Customer Feedback System (C++)

By Pranav Santhosh Nair

01 Problem Statement

Businesses struggle to collect, store, and review customer feedback in an organized way. Without a dedicated system, ratings and comments are scattered, making it impossible to measure overall satisfaction or spot trends.

01

No Data Storage

Customer ratings and messages are never saved, so there's nothing to review after the conversation ends.

02

No Analysis

Without stored data, it's impossible to calculate an average rating or identify satisfied vs unhappy customers.

03

No Structure

Feedback is informal and unstructured — no consistent format for name, rating, or message exists.

02 Our Solution

What We Built

A menu-driven C++ console application that collects customer feedback, stores it using structs and vectors, and generates a summarized report with an average rating.

Input Collection

Collects customer name, rating (1–5), and a feedback message with full input validation.

Data Storage

Uses a Feedback struct stored in a vector to hold unlimited entries during runtime.

Feedback Summary

Displays every entry and calculates the average rating across all responses.

Menu System

A do-while loop presents 3 options: Give Feedback, View Summary, and Exit.

03 Working — Method & Flow

Key Components Used

`struct Feedback`	Groups name, rating, and message into one unit
`vector<Feedback>`	Dynamically stores all feedback entries at runtime
`getline()`	Captures full-sentence messages with spaces
`do-while loop`	Keeps the menu active until the user exits
`Input validation`	Rejects any rating outside the 1–5 range
`setprecision(1)`	Formats the average rating to one decimal place

04 Conclusion

The Customer Feedback System demonstrates how core C++ concepts can be combined to solve a real-world problem — collecting, storing, and analyzing user feedback efficiently.

Problem Solved

Businesses can now collect structured feedback with a name, rating, and message — all in one program.

C++ Skills Applied

Structs, vectors, loops, input validation, and string handling were all applied in a practical context.

Scalable Design

The vector-based storage means the program can handle any number of feedback entries without modification.

Real-World Output

The summary report with average rating gives actionable insight — matching how real feedback tools work.

ITM SKILLS UNIVERSITY

Thank You